

Semana 7: Matemáticas para ML I — álgebra lineal y cálculo esencial

Nivel: Intermedio | **Prerrequisitos:** Semana 6 (funciones probadas con `pytest`), Semana 5 (razonar sobre costo de un algoritmo), Semana 4 (listas y listas de listas), Semana 3 (funciones y bucles) | **Versión:** v1.0 | **Fecha:** 2026-10-05

Objetivos de aprendizaje

1. Representar **vectores** y **matrices** como listas y listas de listas en Python, e interpretar un vector como una lista de características de un dato.
2. Calcular a mano y en código el **producto punto** de dos vectores y el **producto matriz-vector**, y explicar qué significan.
3. Explicar con palabras propias qué es una **derivada** (la pendiente en un punto) y aproximarla numéricamente con una función en Python.
4. Describir y programar el **descenso del gradiente** en una dimensión, y explicar qué papel juega la **tasa de aprendizaje**.
5. Reconocer cómo estas piezas se combinan en una "neurona" simple que aprende un parámetro a partir de ejemplos.

1. Por qué las matemáticas del ML son más simples de lo que parecen

Casi todo el Machine Learning se apoya en dos ideas:

- **Álgebra lineal:** una forma ordenada de manejar muchos números a la vez (cada dato de entrenamiento es una lista de números).
- **Cálculo:** una forma de saber hacia dónde mover un número para que un error disminuya.

Esta semana no se demuestra nada: se construye intuición y se programa cada idea en Python puro, con lo que ya conoces (listas, funciones, bucles). **NumPy** (que hace esto mismo más rápido y con menos código) se introduce en la Semana 9; conviene entender primero qué hace por debajo.

2. Vectores: una lista de características

Analogía: la ficha de un jugador. Un jugador de fútbol puede resumirse con tres números: `[goles, asistencias, minutos]`. Un **vector** es exactamente eso: una lista ordenada de números donde la posición tiene significado.

```
jugador_a = [12, 5, 2700]    # goles, asistencias, minutos
jugador_b = [3, 9, 1800]
```

Operaciones básicas, componente a componente:

```
# Suma de vectores: se suma posición con posición
suma = [x + y for x, y in zip(jugador_a, jugador_b)]    # [15, 14, 4500]

# Multiplicar por un número (escalar): se escala cada posición
doble = [2 * x for x in jugador_a]                    # [24, 10, 5400]
```

(`zip` recorre dos listas en paralelo; las list comprehensions son de la Semana 4.)

En ML, **cada dato es un vector**: una casa son `[metros, habitaciones, antigüedad]`; una imagen en blanco y negro es una lista larga de brillos de píxeles; una palabra puede ser una lista de cientos de números (un embedding).

3. El producto punto: combinar dos vectores en un número

Analogía: la cuenta del supermercado. Si compras `[2, 1, 3]` unidades de tres productos con precios `[4.0, 0.5, 2.0]`, el total es $2 \cdot 4.0 + 1 \cdot 0.5 + 3 \cdot 2.0 = 14.5$. Multiplicar posición con posición y sumar todo es el **producto punto**.

```
def producto_punto(a: list[float], b: list[float]) -> float:
    """Suma de multiplicar componente a componente."""
    if len(a) != len(b):
        # Fallar rápido: un error claro vale más que un resultado silenciosamente
        erróneo.
        raise ValueError("Los vectores deben tener la misma longitud")
    return sum(x * y for x, y in zip(a, b))

print(producto_punto([2, 1, 3], [4.0, 0.5, 2.0]))    # 14.5
```

Qué significa en ML: el producto punto mide cuánto "coinciden" dos vectores. Si ambos apuntan en la misma dirección el resultado es grande y positivo; si son perpendiculares es 0; si apuntan en direcciones opuestas es negativo. Por eso aparece en todas partes: una neurona calcula `entradas · pesos`, y un buscador compara textos midiendo el producto punto (normalizado, lo que se llama similitud coseno) entre sus vectores.

```
print(producto_punto([1, 2, 3], [2, 4, 6]))    # 28 → misma dirección
print(producto_punto([1, 2, 3], [3, 0, -1]))   # 0  → perpendiculares
```

4. Matrices: una tabla de números

Una **matriz** es una tabla: una lista de filas, y cada fila es un vector. En Python, una lista de listas (Semana 4).

```
matriz = [
    [2, 1],
    [0, 3],
]
```

El **producto matriz-vector** hace un producto punto entre cada fila y el vector, y junta los resultados en un vector nuevo:

```
def matriz_por_vector(matriz: list[list[float]], v: list[float]) -> list[float]:
    """Cada fila de la matriz hace un producto punto con el vector."""
    return [producto_punto(fila, v) for fila in matriz]

print(matriz_por_vector([[2, 1], [0, 3]], [1, 2]))    # [4, 6]
```

Paso a paso: fila 1 → $2 \cdot 1 + 1 \cdot 2 = 4$; fila 2 → $0 \cdot 1 + 3 \cdot 2 = 6$.

Qué significa en ML: una capa de una red neuronal es una matriz de pesos; aplicarla a un dato es exactamente `matriz_por_vector`. Una matriz transforma vectores (los gira, estira o combina). Con miles de filas, esta operación es el trabajo principal de una GPU.

Costo (conexión con la Semana 5): `matriz_por_vector` con una matriz de m filas y n columnas hace $m \cdot n$ multiplicaciones, es decir, $O(m \cdot n)$. Un bucle de Python puro es lento para esto; por eso se usará NumPy más adelante.

5. Cálculo: la derivada como pendiente

Analogía: la ladera de una montaña. Si estás parado en una ladera, la **pendiente** te dice qué tan empinado es el suelo justo donde estás y hacia dónde sube. La **derivada** de una función en un punto x es esa pendiente.

- Derivada positiva $\rightarrow$ la función sube al avanzar a la derecha.
- Derivada negativa $\rightarrow$ la función baja.
- Derivada cero $\rightarrow$ estás en una cima o en un valle (un punto plano).

No hace falta derivar con fórmulas: se puede **aproximar** midiendo la pendiente entre dos puntos muy cercanos.

```
def derivada_numerica(f, x: float, h: float = 1e-6) -> float:
    """Pendiente aproximada de f en x (diferencia central)."""
    return (f(x + h) - f(x - h)) / (2 * h)

print(round(derivada_numerica(lambda x: x ** 2, 3), 4))    # 6.0
```

Para $f(x) = x^2$, la regla de derivación da $2x$, que en $x = 3$ es 6 : coincide. (Se pasa una función como argumento — las funciones también son valores en Python.)

6. Descenso del gradiente: bajar la montaña a ciegas

Analogía: bajar un cerro con niebla. No ves el valle, pero sientes la pendiente bajo los pies. La estrategia: da un paso pequeño en dirección contraria a la pendiente (cuesta abajo), y repite. Eventualmente llegas a un valle.

Eso es el **descenso del gradiente**. En una dimensión, el "gradiente" es simplemente la derivada. La regla es:

```
x_nuevo = x - tasa_de_aprendizaje * derivada(x)
```

```
def descenso_gradiente(f, x_inicial: float, tasa: float = 0.1, pasos: int = 50) -> float:
    """Busca un mínimo de f bajando por la pendiente, paso a paso."""
    x = x_inicial
    for _ in range(pasos):
        x = x - tasa * derivada_numerica(f, x)
    return x

# Mínimo de  $(x - 3)^2$ , que sabemos que está en  $x = 3$ 
error = lambda x: (x - 3) ** 2
print(round(descenso_gradiente(error, x_inicial=0), 4)) # 3.0
```

Los primeros pasos desde $x = 0$: $0.6 \rightarrow 1.08 \rightarrow 1.464 \rightarrow 1.7712 \rightarrow 2.017...$ Cada paso es más corto porque la pendiente se aplana al acercarse al valle.

La **tasa de aprendizaje** (`tasa`) controla el tamaño del paso: demasiado pequeña y avanzas con lentitud; demasiado grande y te pasas de largo y puedes divergir (rebotar de un lado al otro de forma cada vez más violenta). Elegirla bien es una de las decisiones más prácticas del ML.

7. Por qué esto importa para IA/ML

- **Entrenar un modelo = minimizar un error.** Se define una función de error (qué tan mal predice el modelo) y se usa descenso del gradiente para ajustar los pesos hasta que el error baje.
- **Los pesos son vectores y matrices;** hacer una predicción es producto punto / matriz-vector.
- **Muchas dimensiones:** con millones de pesos, el gradiente deja de ser un número y pasa a ser un vector (una derivada por peso). La idea es la misma que en una dimensión; los frameworks (PyTorch, TensorFlow, Semana 11) la calculan automáticamente (autodiferenciación).

8. Ejemplo práctico: una neurona que aprende un solo peso

Problema: tenemos ejemplos $(\text{entrada}, \text{salida}) = (1, 2), (2, 4), (3, 6)$. Queremos descubrir el número w tal que $\text{salida} \approx w \cdot \text{entrada}$.

Pseudocódigo

```
w ← 0
repetir 100 veces:
    para cada ejemplo (x, y): error = w·x - y
    gradiente ← promedio de 2 · error · x
    w ← w - tasa · gradiente
mostrar w
```

Código real (Python)

```
datos = [(1, 2), (2, 4), (3, 6)] # (entrada, salida esperada)
w = 0.0 # el peso que debe aprender el modelo
tasa = 0.05

for _ in range(100):
    # Derivada del error cuadrático medio respecto a w, promediada sobre los datos.
    gradiente = sum(2 * (w * x - y) * x for x, y in datos) / len(datos)
    w = w - tasa * gradiente

print(round(w, 4)) # 2.0 → el modelo "descubrió" que salida = 2 · entrada
```

Esto es **entrenar un modelo de ML**, en miniatura: datos, un parámetro, una medida de error, y descenso del gradiente. Todo lo demás del Deep Learning es una versión más grande de este mismo bucle.

Pruebas con pytest (Semana 6)

```
# test_algebra.py
import pytest
from algebra import producto_punto, matriz_por_vector, derivada_numerica,
descenso_gradiente

def test_producto_punto():
    assert producto_punto([1, 2, 3], [4, 5, 6]) == 32

def test_longitudes_distintas():
    with pytest.raises(ValueError):
        producto_punto([1, 2], [1, 2, 3])

def test_matriz_por_vector():
    assert matriz_por_vector([[1, 0], [0, 1]], [7, 9]) == [7, 9]    # la identidad no
cambia nada
    assert matriz_por_vector([[2, 1], [0, 3]], [1, 2]) == [4, 6]

def test_derivada():
    # Con decimales nunca se compara con ==: se usa una tolerancia.
    assert derivada_numerica(lambda x: x ** 2, 3) == pytest.approx(6, abs=1e-4)

def test_descenso():
    assert descenso_gradiente(lambda x: (x - 3) ** 2, 0) == pytest.approx(3,
abs=1e-3)
```

Las cinco pruebas pasan al ejecutar `pytest` sobre las funciones de esta clase (verificado antes de publicar).

9. Notas de vigencia técnica (2026)

- **Python puro vs. NumPy:** en la práctica profesional estas operaciones se hacen con **NumPy** (`np.dot`, el operador `@`), mucho más rápido porque ejecuta código compilado y vectorizado. Esta semana se escribe a mano a propósito, para entender qué hace; NumPy llega en la Semana 9.

- **Derivadas numéricas vs. autodiferenciación:** la aproximación con `h` pequeño es didáctica y sirve para verificar resultados. Ningún framework moderno entrena así: **PyTorch** y **JAX** (y TensorFlow) calculan gradientes exactos y automáticos (autograd). Se verá en la Semana 11.
- **Optimizadores:** el descenso del gradiente simple de esta clase es la base; en la práctica actual se usan variantes como **Adam/AdamW**, que adaptan la tasa de aprendizaje de cada peso. La idea central no cambia.
- **Tipado:** `list[float]` (sin importar `List` de `typing`) es sintaxis estándar desde Python 3.9.

10. Errores comunes de principiante

- Calcular el producto punto de vectores de distinta longitud: `zip` corta en silencio al más corto y devuelve un resultado erróneo sin avisar (por eso la función valida la longitud).
- Confundir el **producto punto** (da un número) con la **suma o multiplicación componente a componente** (da un vector).
- Comparar decimales con `==` en pruebas (`0.1 + 0.2 == 0.3` es `False`); usar `pytest.approx`.
- Restar el gradiente con el signo equivocado (`x + tasa * derivada` sube la montaña en vez de bajarla).
- Elegir una tasa de aprendizaje demasiado grande y ver que el error crece en vez de bajar.

11. Ejercicio propuesto

Escribe una función `similitud_coseno(a, b)` que devuelva `producto_punto(a, b) / (norma(a) * norma(b))`, donde `norma(v)` es la raíz cuadrada del producto punto de `v` consigo mismo (usa `math.sqrt`). Comprueba que `[1, 2, 3]` y `[2, 4, 6]` dan `1.0` (misma dirección) y que `[1, 2, 3]` y `[3, 0, -1]` dan `0.0` (perpendiculares). Escribe al menos tres pruebas con `pytest` y versiona el resultado con Git en dos commits.

Pista: cuida el caso de un vector de ceros (su norma es 0 y dividirías entre cero); decide cómo manejarlo y pruébalo.

12. Autoevaluación

1. ¿Qué es un vector en ML? — Una lista ordenada de números donde cada posición tiene significado; cada dato suele representarse como un vector de características.
2. Calcula el producto punto de `[1, 2, 3]` y `[4, 5, 6]`. — $1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$.

3. ¿Qué mide, intuitivamente, el producto punto? — Cuánto "coinciden" en dirección dos vectores: grande y positivo si apuntan igual, 0 si son perpendiculares, negativo si son opuestos.
4. ¿Qué hace el producto matriz-vector? — Calcula el producto punto de cada fila de la matriz con el vector y junta los resultados en un vector nuevo.
5. ¿Qué indica el signo de la derivada de una función en un punto? — Positiva: la función sube; negativa: baja; cero: punto plano (cima o valle).
6. ¿Por qué en el descenso del gradiente se resta la derivada? — Para moverse en dirección contraria a la pendiente, es decir, hacia donde la función baja.
7. ¿Qué pasa si la tasa de aprendizaje es demasiado grande? — Los pasos se pasan del mínimo y el valor puede oscilar o divergir en vez de converger.
8. V/F: "Con `zip`, calcular el producto punto de vectores de distinta longitud lanza un error automáticamente." — Falso; `zip` corta al más corto sin avisar, por eso se valida la longitud explícitamente.
9. Sin jerga: ¿qué significa "entrenar" un modelo, según lo visto esta semana? — Ajustar números (pesos) repetidamente, dando pasos cuesta abajo en una medida de error, hasta que las predicciones coincidan con los ejemplos.

13. Verificación de dependencias

Depende de la Semana 6 (las funciones se prueban con `pytest`, incluido `pytest.raises` y ahora `pytest.approx`, y el ejercicio se versiona con Git), de la Semana 5 (el análisis $O(m \cdot n)$ del producto matriz-vector), de la Semana 4 (listas, listas de listas y `list comprehensions` para representar vectores y matrices) y de la Semana 3 (funciones, valores por defecto y bucles `for`; además, pasar una función como argumento, ya implícito al usar `key=` en la Semana 4). Todo el cálculo y el álgebra se introducen desde cero aquí, con solo aritmética básica como base; no se usa NumPy (Semana 9) ni ninguna librería externa salvo `pytest`. Es prerequisite directo de la Semana 8 (probabilidad y estadística, que reutiliza vectores y funciones de error) y del Módulo 10 en adelante, donde el descenso del gradiente se usa para entrenar modelos reales.